

Phase 2 API Documentation

Overview

Phase 2 provides three main subsystems with comprehensive APIs:

1. **NUMA Subsystem:** Topology detection, per-CPU arenas, attention-guided allocation
2. **Prefetch Subsystem:** Huge pages, PCID/ASID, TLB management
3. **Scheduler Subsystem:** Timer wheel, tickless operation, CRDS scheduling

NUMA Subsystem

NUMA Topology

```
#include "numa_topology.h"

// Initialize NUMA topology
numa_topology_t* topo = numa_topology_init();

// Get current CPU and node
int cpu = numa_topology_current_cpu();
int node = numa_topology_current_node();

// Get distance between nodes
int dist = numa_topology_distance(node1, node2);

// Find closest node
int closest = numa_topology_closest_node(node);

// Print topology
numa_topology_print();

// Cleanup
numa_topology_destroy();
```

Per-CPU Arena Allocator

```
#include "per_cpu_arena.h"

// Initialize all per-CPU arenas
per_cpu_arena_init();

// Allocate from current CPU's arena
void* ptr = per_cpu_arena_alloc(1024);

// Allocate aligned memory
void* aligned_ptr = per_cpu_arena_alloc_aligned(2048, 64);

// Free memory
per_cpu_arena_free(ptr, 1024);

// Get statistics
per_cpu_arena_stats_t stats;
per_cpu_arena_get_stats(&stats);

// Print statistics
per_cpu_arena_print_stats(cpu_id);

// Cleanup
per_cpu_arena_destroy();
```

Attention-Guided Allocation

```
#include "attention.h"

// Initialize with configuration
attention_config_t config = {
    .migration_threshold = 1000,
    .migration_cooldown = 1000,
    .migration_cost_factor = 0.2,
    .ema_alpha = 0.1,
    .enable_migration = true
};
attention_init(&config);

// Record object access
attention_record_access(class_id, node);

// Get preferred node for object class
int preferred = attention_get_preferred_node(class_id);

// Check if should migrate
uint32_t target_node;
if (attention_should_migrate(class_id, current_node, &target_node)) {
    // Perform migration
    attention_record_migration(class_id, current_node, target_node);
}

// Get statistics
attention_class_stats_t stats;
attention_get_stats(class_id, &stats);

// Cleanup
attention_destroy();
```

Prefetch Subsystem

Huge Pages

```
#include "huge_pages.h"

// Initialize
huge_page_config_t config = {
    .enable_2mb = true,
    .enable_1gb = true,
    .enable_thp = true,
    .promotion_threshold = 512 * 1024,
    .demotion_threshold = 80
};
huge_page_init(&config);

// Allocate 2MB huge page
void* page_2mb = huge_page_alloc(HUGE_PAGE_TYPE_2MB);

// Allocate 1GB huge page
void* page_1gb = huge_page_alloc(HUGE_PAGE_TYPE_1GB);

// Allocate from specific NUMA node
void* page_node = huge_page_alloc_node(HUGE_PAGE_TYPE_2MB, node);

// Promote regular pages to huge page
huge_page_promote(addr, size);

// Demote huge page to regular pages
huge_page_demote(addr, HUGE_PAGE_TYPE_2MB);

// Free huge page
huge_page_free(page_2mb, HUGE_PAGE_TYPE_2MB);

// Get statistics
huge_page_stats_t stats;
huge_page_get_stats(&stats);

// Cleanup
huge_page_destroy();
```

PCID/ASID Management

```
#include "pcid.h"

// Initialize
pcid_config_t config = {
    .enable_pcid = true,
    .enable_invpcid = true,
    .eviction_threshold = 3072
};
pcid_init(&config);

// Allocate PCID for context
uint16_t pcid = pcid_alloc(context_id);

// Get PCID for context
uint16_t existing = pcid_get(context_id);

// Invalidate TLB entry
pcid_invalidate(pcid, virtual_addr);

// Invalidate all entries for PCID
pcid_invalidate_all(pcid);

// Free PCID
pcid_free(pcid);

// Check support
bool supported = pcid_is_supported();
bool invpcid_supported = pcid_is_invpcid_supported();

// Get statistics
pcid_stats_t stats;
pcid_get_stats(&stats);

// Cleanup
pcid_destroy();
```

Scheduler Subsystem

Timer Wheel

```
#include "timer_wheel.h"

// Create timer wheel
timer_wheel_t* wheel = timer_wheel_create();

// Add timer
void callback(void* arg) {
    printf("Timer expired!\n");
}

timer_id_t id = timer_wheel_add(wheel, 100, callback, arg);

// Cancel timer
timer_wheel_cancel(wheel, id);

// Advance time by one tick
uint32_t expired = timer_wheel_tick(wheel);

// Advance to specific time
expired = timer_wheel_advance_to(wheel, target_time_ms);

// Get next expiration
uint64_t next = timer_wheel_next_expiration(wheel);

// Get statistics
timer_wheel_stats_t stats;
timer_wheel_get_stats(wheel, &stats);

// Cleanup
timer_wheel_destroy(wheel);
```

Integration

Phase 2 Initialization

```
#include "phase2_init.h"

// Initialize with default configuration
phase2_init(NULL);

// Initialize with custom configuration
phase2_config_t config = {
    .enable_numa = true,
    .enable_attention = true,
    .attention_threshold = 1000,
    .enable_huge_pages = true,
    .enable_pcid = true,
    .enable_prefetch = true,
    .enable_tickless = true,
    .enable_timer_wheel = true,
    .integrate_phase1 = true
};
phase2_init(&config);

// Check initialization status
if (phase2_is_initialized()) {
    printf("Phase 2 is ready\n");
}

// Print status
phase2_print_status();

// Print all statistics
phase2_print_all_stats();

// Reset all statistics
phase2_reset_all_stats();

// Cleanup
phase2_destroy();
```

Error Handling

All Phase 2 APIs follow consistent error handling:

- Functions returning pointers return `NULL` on failure
- Functions returning integers return `-1` on failure, `0` on success
- Functions returning booleans return `false` on failure/not supported
- Invalid parameters are checked and handled gracefully

Example:

```
void* ptr = per_cpu_arena_alloc(size);
if (ptr == NULL) {
    // Handle allocation failure
    fprintf(stderr, "Allocation failed\n");
    return -1;
}

// Use ptr
per_cpu_arena_free(ptr, size);
```

Thread Safety

- **NUMA Topology:** Thread-safe after initialization
- **Per-CPU Arenas:** Lock-free within single CPU context
- **Attention:** Thread-safe with internal synchronization
- **Huge Pages:** Thread-safe
- **PCID:** Thread-safe with internal synchronization
- **Timer Wheel:** Per-CPU instances, no cross-CPU synchronization needed

Performance Considerations

1. **NUMA Locality:** Always allocate from local node when possible
2. **Arena Reuse:** Reuse allocations to avoid fragmentation
3. **Huge Pages:** Use for large allocations (>512KB)
4. **PCID:** Reduces TLB flush overhead by 70-90%
5. **Timer Wheel:** O(1) operations regardless of timer count

Integration with Phase 1

Phase 2 extends Phase 1 components:

```
// Use NUMA-aware allocation with Phase 1 rings
#include "spsc_ring.h"
#include "per_cpu_arena.h"

// Allocate ring buffer from local NUMA node
void* ring_mem = per_cpu_arena_alloc(ring_size);
spsc_ring_t* ring = spsc_ring_create_from_memory(ring_mem, ring_size);

// Use huge pages for fiber stacks
#include "fiber.h"
#include "huge_pages.h"

void* stack = huge_page_alloc(HUGE_PAGE_TYPE_2MB);
fiber_t* fiber = fiber_create_with_stack(entry, arg, stack, HUGE_PAGE_2MB);
```

Examples

See `tests/` directory for comprehensive examples of each API.

Further Reading

- [ARCHITECTURE.md](#) (ARCHITECTURE.md): Detailed architecture
- [PERFORMANCE.md](#) (PERFORMANCE.md): Performance analysis
- [INTEGRATION.md](#) (INTEGRATION.md): Integration guide